

ODE Solver

Mariam Helal

CIERA REU 2026

July 14, 2026

The full code for this ODE solver can be found here: 

Set-up

An ODE solver is an algorithm that can be used to calculate how a system changes with a variable, such as time. A simple set-up for such a solver is a particle in a 3D box. This particle exists in a defined velocity field. The purpose of the ODE solver is to determine, with each time step, what the position of the particle is.

In order to easily access defined functions, it is useful to first create a class based on inputted grid parameters:

```
class ODE_solver:
    def __init__(self, box_limits, grid_shape):
        self.box_limits = np.array(box_limits)
        self.grid_shape = grid_shape
        self.x_grid = np.linspace(box_limits[0][0], box_limits[0][1],
            ↪ grid_shape[0])
        self.y_grid = np.linspace(box_limits[1][0], box_limits[1][1],
            ↪ grid_shape[1])
        self.z_grid = np.linspace(box_limits[2][0], box_limits[2][1],
            ↪ grid_shape[2])
```

Space is discretised for plotting purposes later on. Next, the velocity field must be defined within the **ODE_solver** class. This velocity function takes as its inputs both the current time and position of the particle, and returns the new velocity:

```
def get_velocity(self, pos, t):
    x, y, z = pos[0], pos[1], pos[2]
    u = -y
```

```

v = x
w = 0.5
return np.array([u, v, w])

```

Defining a 3D box for the particle to reside within presents a problem: what happens if the particle's calculated position places it outside the limits of the box? To solve this, a **bounce** function is defined that mirrors the ball's position back inside the box. However, this will cause the ball to simply push against the wall. In order for a true bounce to occur, the velocity must be reversed, which requires making velocity a state variable like position. In the case of the simple set-up presented at the beginning, though, the ball pushing against the wall will suffice, and more complexity can be added afterwards. The bounce function preforms the above task as seen below:

```

def bounce(self, pos):
    for i in range(3):
        min_pos, max_pos = self.box_limits[i]
        if pos[i] > max_pos:
            pos[i] = max_pos - (pos[i] - max_pos)
        elif pos[i] < min_pos:
            pos[i] = min_pos + (min_pos - pos[i])
    return pos

```

The last piece of code this ODE solver requires is the actual calculation to solve the ODE and determine the particle's position. The numerical method used is the Runge-Katta 4th order method, which solves ODEs using the following equations:

$$k_1 = v(x_n, t_n), \quad (1)$$

$$k_2 = v \left(x_n + \frac{\Delta t}{2} k_1, t_n + \frac{\Delta t}{2} \right), \quad (2)$$

$$k_3 = v \left(x_n + \frac{\Delta t}{2} k_2, t_n + \frac{\Delta t}{2} \right), \quad (3)$$

$$k_4 = v(x_n + \Delta t k_3, t_n + \Delta t), \quad (4)$$

$$x_{n+1} = x_n + \frac{\Delta t}{6} (k_1 + 2k_2 + 2k_3 + k_4). \quad (5)$$

where Δt is the time step and x_{n+1} is the new current position. This calculation is done within a solve function in the **ODE_solver** class:

```

def solve(self, initial_pos, t_start, t_end, dt):

```

```

time_steps = np.arange(t_start, t_end, dt)
trajectory = [np.array(initial_pos)]
current_pos = np.array(initial_pos, dtype=float)

for t in time_steps:
    k1 = self.get_velocity(current_pos, t)
    k2 = self.get_velocity(current_pos+0.5*dt*k1, t+0.5*dt)
    k3 = self.get_velocity(current_pos+0.5*dt*k2, t+0.5*dt)
    k4 = self.get_velocity(current_pos+dt*k3, t+dt)

    current_pos += (dt/6)*(k1+2*k2+2*k3+k4)
    current_pos = self.bounce(current_pos)
    trajectory.append(current_pos.copy())
return np.array(trajectory)

```

A trajectory array is created in order to keep track of the particles positions through time, and the bounce function is called to check if the particle's position extends beyond the box limits.

Usage

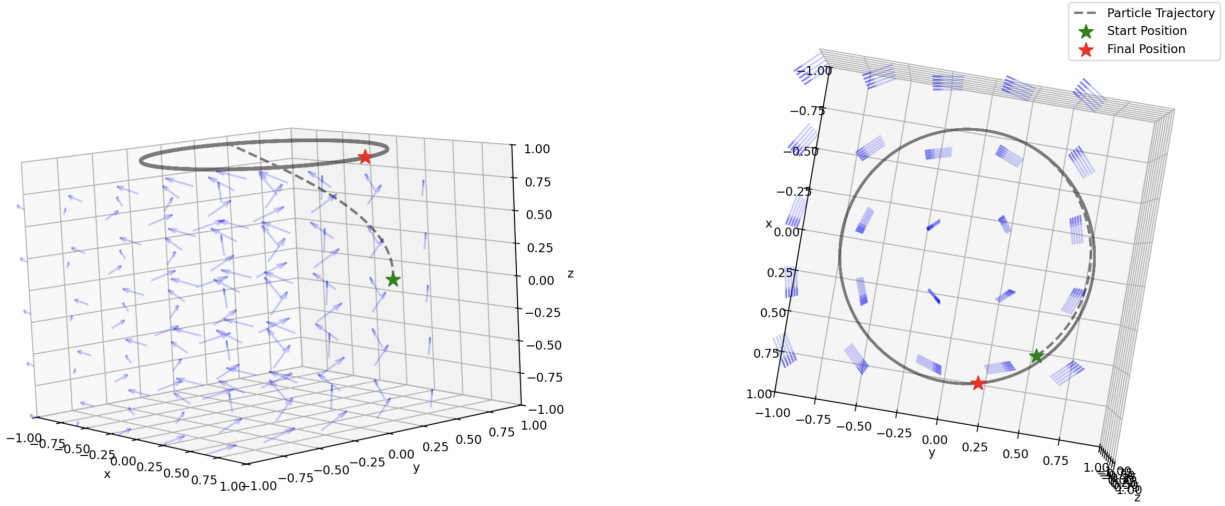
An example of the created ODE solver being used can be seen below:

```

box = [(-1, 1), (-1, 1), (-1, 1)]
grid_res = (50, 50, 50)
solver = ODE_solver(box_limits=box, grid_shape=grid_res)
start_position = [0.5, 0.5, 0]
t0, tf, step_size = 0, 100, 0.02
path = solver.solve(start_position, t0, tf, step_size)

```

The trajectory of the ball can then be plotted, alongside the velocity field (the blue arrows), producing the plots seen in Figure 1:



(a) Side view

(b) Top view

Figure 1: Side and top views showing the ball's trajectory and velocity field